



Documentation Companion Agent (v1.00) - PRD

1. Document Control

Field	Value
Product	Documentation Companion Agent
Version	v1.00
Document Type	Product Requirements Document (PRD)
Status	Draft for Review
Primary Audience	Product, Engineering, QA, R&D
Language	English
Last Updated	2026-04-27
Owner	Coject R&D Team

2. Executive Summary

Documentation Companion Agent v1.00 defines a standalone AI-powered documentation workspace for Coject product teams.

The product allows developers to explain a product or feature through a guided AI interview while the system generates a living Markdown guide, observes the product through a browser when possible, collects screenshots and assets, and flags missing or inconsistent product behavior before client or administration presentations.

The first implementation and demo target is Board Agent.

3. Problem Statement

Coject Workshop contains multiple products and features. Product development teams can build strong technical features, but they are not always prepared to create clear user guides, demo scripts, onboarding material, or client-facing documentation.

Today, feature explanation often depends on live developer presentations. This creates repeated effort, inconsistent presentation quality, missed details, and late discovery of unclear or incomplete product

behavior.

The Documentation Companion Agent must make product explanation repeatable, reviewable, and reusable without requiring developers to repeatedly join live walkthroughs.

4. Product Goal and Success Criteria

Goal

Build a standalone documentation workspace where developers collaborate with an AI agent to produce trusted, reviewable Markdown documentation packages for Coject products and features.

The generated guide should allow a product owner to understand, use, and present the documented product without developer help for approximately 95% of the intended workflow.

Success Criteria

1. A developer can create a documentation session for Board Agent.
 2. The agent can interview the developer about product purpose, target users, access, permissions, navigation, workflows, edge cases, and demo requirements.
 3. The agent can generate and update a structured Markdown guide while the developer works.
 4. The developer can edit the Markdown canvas directly, and the agent can use those edits in later chat context.
 5. The agent can interact with a product URL through a browser when available.
 6. The agent can record browser walkthrough evidence, including screens visited, actions performed, visible labels, validation messages, and screenshots or asset references.
 7. The generated documentation separates developer explanation, observed product behavior, and published documentation.
 8. The guide includes a Product Gaps / Red Flags section for missing answers, skipped questions, uncertainty, and mismatches between expected and observed behavior.
 9. The developer can mark a document complete.
 10. The Coject R&D team can review and approve a completed document.
 11. Approved documents are readonly.
 12. Any change to approved content creates a new draft version.
 13. A product owner can use the approved Board Agent guide and assets to complete the main workflow and present the product without developer support for approximately 95% of the intended workflow.
-

5. Scope (v1.00)

In Scope

1. Standalone Documentation Companion Agent workspace.
2. Product and feature setup for a documentation session.
3. Board Agent as the first demo product.
4. Split-screen workspace:
 - chat with Documentation Companion Agent
 - live Markdown document canvas
5. Chat-based developer interview.
6. Agent follow-up questions that challenge vague or incomplete answers.
7. Direct Markdown canvas editing.
8. Synchronization between chat context and edited Markdown content.
9. Browser interaction with a product URL when available.
10. Browser walkthrough evidence capture.
11. Screenshot and asset collection with stable Markdown references.
12. Structured internal knowledge model used to generate Markdown.
13. Product Gaps / Red Flags section.
14. Mark as complete action.
15. R&D review and approval workflow.
16. Readonly approved versions.
17. New draft version creation when approved content needs changes.
18. English-only generated guide.
19. Markdown guide output with metadata.

Out of Scope

1. Arabic documentation generation.
 2. Multilingual documentation generation.
 3. Public documentation publishing portal.
 4. Advanced permission model beyond R&D approval.
 5. Automatic video generation.
 6. Deep codebase analysis as a dependency for v1.00.
 7. Complex multi-product documentation portal.
 8. Board/Card lifecycle integration.
 9. Full documentation analytics.
 10. Multiple export formats beyond Markdown.
-

6. Target Users

Primary User

- Product development team

The product development team provides implementation details, product behavior, user flows, limitations, screenshots, and answers to the agent's questions.

Secondary User

- Coject R&D team

The Coject R&D team reviews, approves, and uses generated documentation as a trusted source for product understanding and presentation.

Downstream User

- Product owner or presenter

The product owner or presenter uses the approved guide to understand the product, operate the main workflows, present the product, handle common issues, and identify known limitations.

7. User Stories

1. As a developer, I want to create a documentation session for a product, so that I can start documenting a feature in a structured workspace.
2. As a developer, I want to enter the product name, feature name, target audience, and product URL, so that the agent has the starting context for the guide.
3. As a developer, I want the agent to ask targeted questions, so that I do not have to design the guide structure myself.
4. As a developer, I want the agent to challenge vague answers, so that the documentation becomes specific enough for non-developers.
5. As a developer, I want to see a live Markdown guide while chatting, so that I can understand how my answers affect the document.
6. As a developer, I want to directly edit the Markdown canvas, so that I can quickly correct wording or structure without using chat for every change.
7. As a developer, I want the agent to understand my direct Markdown edits, so that future chat updates do not overwrite or ignore my changes.
8. As a developer, I want the agent to ask about roles and permissions, so that the guide explains who can perform each action.
9. As a developer, I want the agent to ask about navigation paths, so that users know where to start in the product.
10. As a developer, I want the agent to ask about success results, so that the guide explains how users know an action is complete.
11. As a developer, I want the agent to ask about edge cases and errors, so that the guide can support real product usage.
12. As a developer, I want the agent to mark skipped or unknown answers, so that unresolved documentation gaps stay visible.
13. As a developer, I want the agent to open the product URL, so that it can compare my explanation against the actual product behavior.
14. As a developer, I want the agent to navigate through product screens, so that the generated guide is grounded in observed behavior.
15. As a developer, I want the agent to click buttons, follow links, and fill forms when instructed, so that it can document real workflows.
16. As a developer, I want the agent to observe labels, routes, dialogs, validation messages, and state changes, so that the documentation reflects the actual interface.
17. As a developer, I want the agent to capture screenshots or asset references, so that the guide can include visual evidence.
18. As a developer, I want the system to store assets in a structured folder, so that Markdown links remain stable across versions.
19. As a developer, I want the agent to identify mismatches between my explanation and observed behavior, so that product issues are discovered before presentations.
20. As a developer, I want the guide to include a Product Gaps / Red Flags section, so that risks and unresolved questions are not hidden.

21. As a developer, I want to mark documentation complete, so that it can move from drafting to review.
 22. As an R&D reviewer, I want to review completed documentation, so that only trusted material becomes approved.
 23. As an R&D reviewer, I want to approve a document, so that the approved guide becomes the source of truth for demos and onboarding.
 24. As an R&D reviewer, I want approved documents to be readonly, so that approved guidance is not changed accidentally.
 25. As a developer, I want changes to approved content to create a new draft version, so that the approved version remains stable.
 26. As a product owner, I want a clear overview of the product, so that I understand what the product does before using it.
 27. As a product owner, I want access and permission instructions, so that I know whether I can use the product and what role is required.
 28. As a product owner, I want step-by-step usage instructions, so that I can complete the main workflow without developer help.
 29. As a product owner, I want common errors and edge cases documented, so that I can recover from expected issues.
 30. As a product owner, I want a demo script, so that I can present the product consistently to clients or administration.
 31. As a product owner, I want known limitations listed, so that I do not overpromise product behavior during a presentation.
 32. As a presenter, I want repeatable demo steps, so that presentation quality does not depend on improvisation.
 33. As a presenter, I want the guide to identify red flags, so that I can decide whether the product is ready for a client-facing demo.
 34. As a QA or product reviewer, I want browser walkthrough evidence, so that I can verify what the agent observed.
 35. As a Coject stakeholder, I want reusable product knowledge, so that guidance, onboarding, demos, and product review all use the same trusted source.
-

8. Functional Requirements

FR-01 Documentation Session Setup

1. The system must allow a developer to create a documentation session.
2. The session must capture product name, feature name, target audience, language, and product URL.
3. The system must support Board Agent as the first demo product.
4. The session must start in Draft state.
5. The system must create a structured working area for the Markdown guide, assets, metadata, questions, and approval state.

FR-02 Split-Screen Documentation Workspace

1. The workspace must display chat and Markdown canvas side by side.
2. The chat panel must allow the developer to answer agent questions and request document updates.
3. The Markdown canvas must show the generated guide live.
4. The Markdown canvas must support direct editing.
5. The chat and document context must remain synchronized after direct edits.
6. The workspace must make red flags visible in the document.

FR-03 Agent Interview Behavior

1. The agent must ask targeted questions about product purpose, users, roles, permissions, navigation, main workflows, success results, errors, limitations, and demo needs.
2. The agent must challenge vague answers.
3. If a developer says a user can "manage" an item, the agent must ask which actions are included.
4. If permissions are unclear, the agent must ask which roles can perform the action.
5. If an answer is skipped or unknown, the system must record that uncertainty.
6. The agent must continue interviewing until required guide sections are complete or explicitly marked unknown.

FR-04 Markdown Guide Generation

1. The system must generate a Markdown guide from structured internal knowledge.
2. Markdown must be treated as the presentation layer, not the only source of truth.
3. The guide must follow the required guide template:

- Product / Feature User Guide
- Document Status
- Version History
- Overview
- Product Purpose
- Target Users
- Access And Permissions
- Navigation Path
- Main User Journey
- Step-By-Step Usage
- Browser Walkthrough Evidence
- Expected System Behavior
- Edge Cases And Errors
- Product Gaps / Red Flags
- Screenshots And Assets
- FAQ
- Demo Script
- Approval

4. The guide must be editable by the developer before approval.

5. The agent must be able to update specific guide sections through chat instructions.

FR-05 Browser Interaction

1. The agent must be able to open the configured product URL when browser access is available.
2. The agent must be able to navigate through the product.
3. The agent must be able to click buttons and links.
4. The agent must be able to fill forms when instructed.
5. The agent must observe screens, labels, routes, buttons, dialogs, errors, validation messages, and state changes.
6. The agent must compare observed behavior with developer-described behavior.
7. The agent must ask follow-up questions when observed behavior is unclear.
8. The agent must capture or reference screenshots as documentation assets.

FR-06 Browser Walkthrough Evidence

1. The guide must include a Browser Walkthrough Evidence section.
2. The section must record the product URL or environment used.
3. The section must record screens visited.
4. The section must record actions performed.
5. The section must record visible labels and controls.
6. The section must record observed validation messages.
7. The section must reference captured screenshots or assets when available.

FR-07 Product Gaps and Red Flags

1. The guide must include a Product Gaps / Red Flags section.
2. The section must record missing answers.
3. The section must record skipped questions.
4. The section must record uncertainty.
5. The section must record expected behavior that is not implemented.
6. The section must record mismatches between developer explanation and observed product behavior.
7. Each red flag should include the expected behavior, observed behavior, and recommended action when available.

FR-08 Documentation Package and Assets

1. The output must be a documentation package, not only a single text blob.
2. The package must support product-specific folders.
3. The package must support version-specific folders.
4. Each version must include a Markdown guide.
5. Each version must include an assets folder.
6. Each version must include metadata.
7. Assets must be referenced from Markdown using stable relative links.

Recommended structure:

```
documentation-workspace/  
  products/  
    board-agent/  
      v1.0/  
        guide.md  
        assets/  
          board-list.png  
          create-board-form.png  
          permission-error.png  
          metadata.json
```

FR-09 Metadata

1. Each documentation version must include metadata.
2. Metadata must include product, version, status, language, approval owner, approval timestamp, source, and coverage target.
3. Metadata values that are not known must be marked as **TBD**.

Recommended metadata:

```
{
  "product": "Board Agent",
  "version": "1.0",
  "status": "approved",
  "language": "en",
  "approvedBy": "Coject R&D Team",
  "approvedAt": "TBD",
  "source": "Documentation Companion Agent",
  "coverageTarget": "95% product-owner self-service"
}
```

FR-10 Completion, Review, Approval, and Versioning

1. The user chatting with the agent must explicitly mark the document as complete.
2. The agent must not independently decide that documentation is complete.
3. Complete documents must be reviewable by the Coject R&D team.
4. The Coject R&D team owns approval.
5. Approved documents must be readonly.
6. If changes are needed after approval, the system must create a new draft version.
7. The system must support the following document states:

- Draft
- Complete
- In Review
- Approved
- Superseded
- Archived

FR-11 Demo Script

1. The generated guide must include a Demo Script section.
 2. The demo script must provide a repeatable presentation flow.
 3. The demo script must identify what to show, what to explain, and which known limitations or red flags to mention.
 4. The Board Agent demo script must support client or administration presentation without relying on developer improvisation.
-

9. Non-Functional Requirements

NFR-01 Usability

1. The workspace must behave like a documentation workspace, not only a chat window.
2. Developers must be able to understand the current document status while working.
3. Product gaps and red flags must be visible and easy to review.
4. Product owners must be able to follow approved guides without developer help for approximately 95% of the intended workflow.

NFR-02 Traceability

1. The system must preserve the distinction between developer explanation, observed product behavior, and published documentation.
2. Browser walkthrough evidence must be traceable to documented steps.
3. Skipped answers and unknowns must remain visible until resolved or explicitly accepted.

NFR-03 Maintainability

1. Structured knowledge must be stored separately from Markdown rendering.
2. Guide sections must be generated from a stable internal model.
3. Versioned documentation packages must avoid unstable asset links.
4. The product should support future extension to additional products beyond Board Agent.

NFR-04 Reliability

1. Direct Markdown edits must not be silently lost after chat-based updates.
2. Approved documents must remain stable and readonly.
3. Browser observation failures must be recorded as documentation gaps instead of silently ignored.

NFR-05 Governance

1. Approval must remain owned by the Coject R&D team.
2. The system must not publish approved documentation without R&D approval.
3. Version state transitions must be explicit and auditable.

10. Architecture and Interface Contracts

Required Modules

1. Documentation Session module:

- creates documentation sessions
- stores product, feature, audience, URL, language, status, and version context

2. Agent Interview module:

- manages question flow
- tracks answered, skipped, and unresolved questions
- produces structured knowledge updates

3. Markdown Canvas module:

- renders the generated guide
- supports direct editing
- synchronizes edited content back into agent context

4. Browser Walkthrough module:

- opens product URLs
- performs instructed browser actions
- records observed behavior
- captures or references screenshots

5. Structured Knowledge module:

- stores product facts, workflows, permissions, evidence, assets, red flags, questions, and approval state
- generates Markdown guide content from structured data

6. Asset Management module:

- stores screenshots and supporting assets in versioned folders
- maintains stable relative Markdown links

7. Review and Approval module:

- handles document completion
- supports R&D review
- locks approved versions
- creates new draft versions after approval

Required State Model

The system must support the following minimum internal structure:

```
{
  "product": "Board Agent",
  "feature": "TBD",
  "audience": "Product Owner",
  "language": "en",
  "status": "draft",
  "sections": {
    "overview": {},
    "permissions": {},
    "navigationPath": {},
    "userJourney": {},
    "stepByStepUsage": {},
    "browserEvidence": {},
    "edgeCases": {},
    "redFlags": {},
    "demoScript": {}
  },
  "assets": [],
  "questions": [],
  "approval": {
    "state": "pending",
    "reviewer": "Coject R&D Team"
  }
}
```

Key Product Principle

The system must always separate:

Developer explanation
Observed product behavior
Published documentation

This separation is required to make the generated guide trustworthy and to turn documentation into a product-quality review workflow.

11. Board Agent Demo Scenario

The first v1.00 demo must show the complete documentation workflow using Board Agent.

Required demo flow:

1. Create a new documentation session for Board Agent.
 2. Enter product name, feature name, audience, and product URL.
 3. Start the developer interview.
 4. Let the agent ask questions about purpose, users, navigation, permissions, and main flow.
 5. Use browser interaction to open Board Agent and walk through the main flow.
 6. Let the agent capture observed behavior and identify any mismatches.
 7. Generate the Markdown guide in the right-side canvas.
 8. Directly edit one section in the canvas.
 9. Ask the agent to update another section through chat.
 10. Confirm that both editing paths stay aligned.
 11. Mark the document as complete.
 12. Send it to R&D approval.
 13. Approve the guide and create readonly version `v1.0`.
 14. Attempt a modification after approval and show that it creates a new draft version.
-

12. Testing and Quality Gates

Testing Principles

1. Tests should validate external behavior, not internal implementation details.
2. Tests should confirm that documentation state transitions are correct.
3. Tests should confirm that direct edits and chat updates remain synchronized.
4. Tests should confirm that red flags are created when information is missing or observed behavior conflicts with developer explanation.
5. Tests should confirm that approved documents cannot be modified in place.

Automated Baseline

1. Unit tests for structured knowledge updates and Markdown generation.
2. Unit tests for document state transitions.
3. Unit tests for version creation after approved content changes.
4. Integration tests for chat-driven guide updates.
5. Integration tests for direct Markdown edits syncing back into agent context.
6. Integration tests for browser evidence records.
7. Integration tests for asset references in generated Markdown.
8. Integration tests for approval and readonly behavior.

Manual Smoke Suite

1. Create a Board Agent documentation session.
2. Complete a guided interview.
3. Generate a Markdown guide.
4. Directly edit the guide and confirm agent awareness.
5. Use chat to update a guide section.
6. Run a browser walkthrough against a product URL.
7. Confirm screenshots or assets are stored and referenced.
8. Confirm skipped answers appear as red flags.
9. Confirm mismatched observed behavior appears as a red flag.
10. Mark documentation complete.
11. Move documentation into review.
12. Approve documentation as Coject R&D.
13. Confirm approved guide is readonly.
14. Attempt post-approval modification and confirm a new draft version is created.

Acceptance Criteria

1. Board Agent guide can be produced from a developer interview and browser walkthrough.
2. Guide follows the required Markdown guide template.
3. Guide includes browser walkthrough evidence.
4. Guide includes Product Gaps / Red Flags.
5. Guide includes a repeatable demo script.
6. Guide assets use stable relative links.
7. Direct Markdown edits remain synchronized with chat context.
8. Approved documentation is readonly.
9. New changes after approval create a new draft version.
10. Product owner can use the generated Board Agent guide for approximately 95% self-service workflow completion.

13. Risks and Mitigations

R-01 Vague Developer Input

- Risk: developers may provide incomplete or overly technical answers.
- Mitigation: agent must challenge vague answers and record skipped or unknown answers as red flags.

R-02 Browser Observation Gaps

- Risk: the agent may not be able to fully access or operate the product URL.
- Mitigation: record failed observations as browser evidence gaps and ask the developer for confirmation.

R-03 Documentation Drift

- Risk: generated documentation may become outdated as the product changes.
- Mitigation: use versioned documentation packages and require new draft versions for changes after approval.

R-04 Approval Bypass

- Risk: unreviewed documentation may be treated as official.
- Mitigation: approved state must be explicit, R&D-owned, and readonly.

R-05 Asset Link Instability

- Risk: screenshots and references may break across versions.
- Mitigation: store assets in structured version folders and use stable relative Markdown links.

14. Assumptions

1. Board Agent is available as the first demo product.
 2. The development team can provide product URL access for browser walkthroughs where possible.
 3. The initial release supports English-only documentation.
 4. Coject R&D team is the approval owner for v1.00.
 5. Markdown is the primary output format for v1.00.
 6. Browser interaction is used to strengthen evidence, but the system can still record gaps when browser interaction is unavailable.
 7. Product owner self-service target is approximately 95% of the intended documented workflow, not every possible product scenario.
-

15. Required Companion Docs

1. Technical Architecture Document (v1.00)
 2. Agent Interview Question Model (v1.00)
 3. Markdown Guide Template Specification (v1.00)
 4. Browser Walkthrough Evidence Contract (v1.00)
 5. Documentation Package and Metadata Contract (v1.00)
 6. Review and Approval Workflow Specification (v1.00)
 7. Board Agent Demo Script and Smoke Checklist (v1.00)
-